

DATA-260 Homework 4

Name: Ayushi Shringi

SID4: 0036

DOMAIN_ID: 4

Domain Entity: Open-source package vulnerabilities

Port: 8036

Prefix: s0036

SEED: 0036

VERIFY_SEED: 260036

Python Version: Python 3.12.14

Embedding Model: sentence-transformers/all-MiniLM-L6-v2

Hardware: MacBook Pro

Chip: Apple M5 Pro

Memory: 24 GB

Operating System: macOS Tahoe 26.5.1

GitHub Repository: <https://github.com/ayushishringi/data260-0036/tree/hw4>

Code Commit Hash

Git Tag: hw4

MySQL table and

```
database.py × schemas.py × main.py × Home.jsx × seed_hw4_n

backend > database.py > Base
1  import os
2  from collections.abc import Generator
3
4  from sqlalchemy import create_engine
5  from sqlalchemy.orm import DeclarativeBase, Session, sessionmaker
6
7
8  DATABASE_URL = os.getenv(
9      "DATABASE_URL",
10     "mysql+pymysql://root@127.0.0.1:3306/s0036_rel",
11 )
12
13
14 engine = create_engine(
15     DATABASE_URL,
16     pool_pre_ping=True,
17 )
18
19
20 class Base(DeclarativeBase):
21     pass
22
23
24 db_session_basedede26 = sessionmaker(
25     bind=engine,
26     autoflush=False,
27     autocommit=False,
28 )
29
30
31 def get_db() -> Generator[Session, None, None]:
32     db = db_session_basedede26()
33
34     try:
35         yield db
36     finally:
37         db.close()
```

```

8
9 class User(Base):
10     __tablename__ = "users"
11
12     id: Mapped[int] = mapped_column(
13         Integer,
14         primary_key=True,
15         autoincrement=True,
16     )
17
18     name: Mapped[str] = mapped_column(
19         String(120),
20         nullable=False,
21     )
22
23     email: Mapped[str] = mapped_column(
24         String(255),
25         unique=True,
26         nullable=False,
27     )
28
29     password_hash: Mapped[str] = mapped_column(
30         String(255),
31         nullable=False,
32     )
33
34     sessions: Mapped[list["LoginSession"]] = relationship(
35         back_populates="user",
36         cascade="all, delete-orphan",
37     )
38
39
40 class LoginSession(Base):
41     __tablename__ = "sessions"
42
43     id: Mapped[str] = mapped_column(
44         String(128),
45         primary_key=True,
46     )
47
48     user_id: Mapped[int] = mapped_column(
49         ForeignKey("users.id"),
50         nullable=False,
51     )
52
53     created_at: Mapped[datetime] = mapped_column(

```

backend > models.py > RelatedAdvisory

```
68 class VulnerabilityReport(Base):
69
70     )
71
72     affected_version: Mapped[str] = mapped_column(
73         String(255),
74         nullable=False,
75     )
76
77     submitter_email: Mapped[str] = mapped_column(
78         String(255),
79         nullable=False,
80     )
81
82     description: Mapped[str] = mapped_column(
83         Text,
84         nullable=False,
85     )
86
87     severity: Mapped[str] = mapped_column(
88         String(20),
89         nullable=False,
90     )
91
92     agreed_to_terms: Mapped[bool] = mapped_column(
93         Boolean,
94         nullable=False,
95     )
96
97     submission_date: Mapped[datetime] = mapped_column(
98         DateTime,
99         nullable=False,
100     )
101
102     advisories: Mapped[list["RelatedAdvisory"]] = relationship(
103         back_populates="report",
104         cascade="all, delete-orphan",
105     )
106
107
108 class RelatedAdvisory(Base):
109     __tablename__ = "related_advisories"
110
111     id: Mapped[int] = mapped_column(
112         Integer,
113         primary_key=True,
114         autoincrement=True,
115     )
116
117     report_id: Mapped[int] = mapped_column(
```



```

63         db.commit()
64
65         return session_token
66
67
68     def get_current_user(
69         session_token: str | None = Cookie(
70             default=None,
71             alias=SESSION_COOKIE_NAME,
72         ),
73         db: Session = Depends(get_db),
74     ) -> User:
75         if not session_token:
76             raise HTTPException(
77                 status_code=401,
78                 detail="Login required",
79             )
80
81         current_time = datetime.now(timezone.utc).replace(tzinfo=None)
82
83         database_session = db.scalar(
84             select(LoginSession).where(
85                 LoginSession.id == session_token,
86                 LoginSession.expires_at > current_time,
87             )
88         )
89
90         if database_session is None:
91             raise HTTPException(
92                 status_code=401,
93                 detail="Login required",
94             )
95
96         user = db.get(User, database_session.user_id)
97
98         if user is None:
99             raise HTTPException(
100                 status_code=401,
101                 detail="Login required",
102             )
103
104         return user

```

```

23 @router.post("/register", status_code=201)
24 def register(
25     payload: RegisterRequest,
26     db: Session = Depends(get_db),
27 ) -> dict:
28     existing_user = db.scalar(
29         select(User).where(User.email == payload.email)
30     )
31
32     if existing_user is not None:
33         raise HTTPException(
34             status_code=409,
35             detail="Email already registered",
36         )
37
38     user = User(
39         name=payload.name.strip(),
40         email=str(payload.email),
41         password_hash=hash_password(payload.password),
42     )
43
44     db.add(user)
45     db.commit()
46     db.refresh(user)
47
48     return {
49         "id": user.id,
50         "name": user.name,
51         "email": user.email,
52     }
53
54
55 @router.post("/login")
56 def login(
57     payload: LoginRequest,
58     response: Response,
59     db: Session = Depends(get_db),
60 ) -> dict:
61     user = db.scalar(
62         select(User).where(User.email == payload.email)
63     )
64
65     if user is None or not verify_password(
66         payload.password,
67         user.password_hash,
68     ):
69         raise HTTPException(
70             status_code=401

```

```

56 def login(
57     raise HTTPException(
58         status_code=401,
59         detail="Invalid email or password",
60     )
61
62     session_token = create_server_session(db, user)
63
64     response.set_cookie(
65         key=SESSION_COOKIE_NAME,
66         value=session_token,
67         httponly=True,
68         samesite="lax",
69         secure=False,
70         max_age=1800,
71     )
72
73     return {
74         "id": user.id,
75         "name": user.name,
76         "email": user.email,
77     }
78
79 @router.get("/me")
80 def get_me(
81     current_user: User = Depends(get_current_user),
82 ) -> dict:
83     return {
84         "id": current_user.id,
85         "name": current_user.name,
86         "email": current_user.email,
87     }
88
89 @router.post("/logout", status_code=204)
90 def logout(
91     response: Response,
92     session_token: str | None = Cookie(
93         default=None,
94         alias=SESSION_COOKIE_NAME,
95     ),
96     current_user: User = Depends(get_current_user),
97     db: Session = Depends(get_db),
98 ) -> None:
99     if session_token is not None:
100         database_session = db.get(
101             LoginSession

```

```

● ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % mysql -u root -D s0036_rel -e "SHOW TABLES;"
+-----+
| Tables_in_s0036_rel |
+-----+
| related_advisories  |
| sessions            |
| users               |
| vulnerability_reports |
+-----+

```

MySQL table and stored application data

```

● ((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % mysql -u root -D s0036_rel -e "
SELECT COUNT(*) AS users FROM users;
SELECT COUNT(*) AS sessions FROM sessions;
SELECT COUNT(*) AS reports FROM vulnerability_reports;
"
+-----+
| users |
+-----+
|      1 |
+-----+
+-----+
| sessions |
+-----+
|       15 |
+-----+
+-----+
| reports |
+-----+
|      5000 |
+-----+

```

Login Page

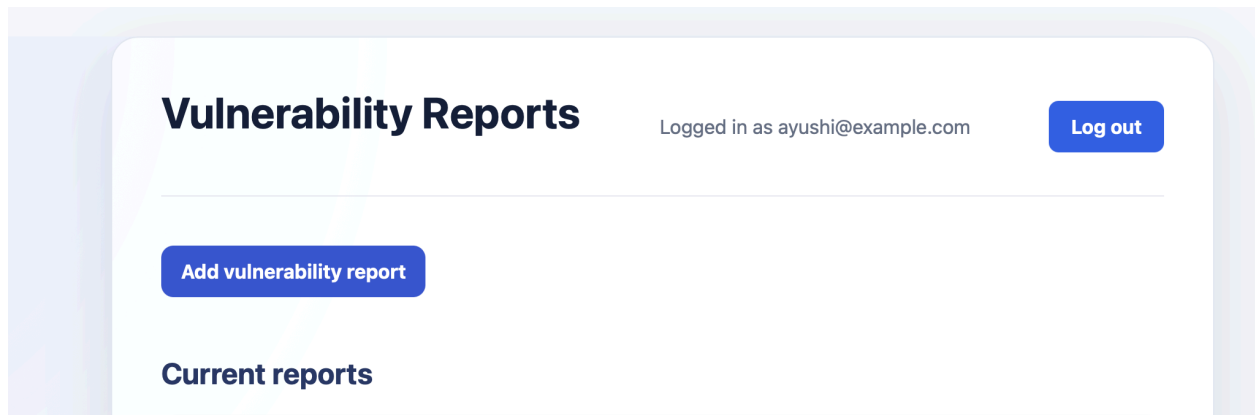
Login

Email

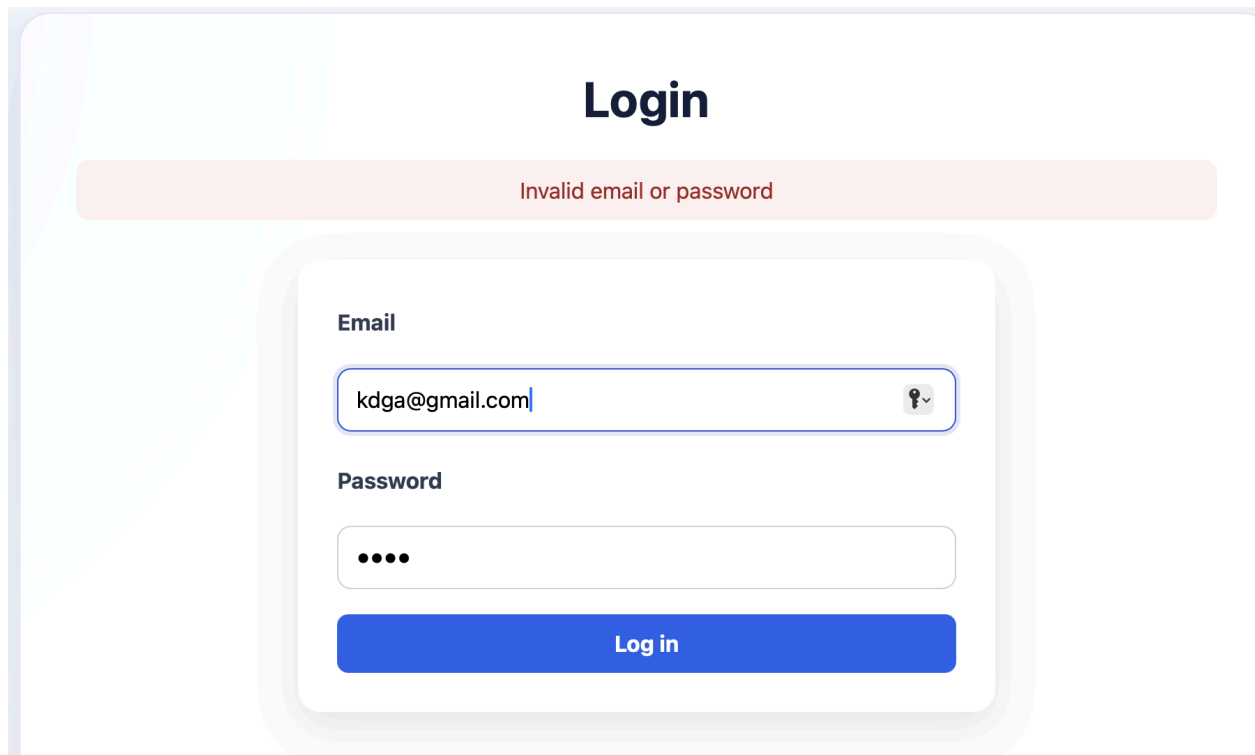
Password

[Log in](#)

Home Page



Invalid Login



React CRUD Interface

```

32     }
33   />
34
35   <Route
36     path="/"
37     element={
38       user ? (
39         <Home user={user} onLogout={handleLogout} />
40       ) : (
41         <Navigate to="/login" replace />
42       )
43     }
44   />
45
46   <Route
47     path="/create"
48     element={
49       user ? (
50         <CreateRecord />
51       ) : (
52         <Navigate to="/login" replace />
53       )
54     }
55   />
56
57   <Route
58     path="/update/:id"
59     element={
60       user ? (
61         <UpdateRecord />
62       ) : (
63         <Navigate to="/login" replace />
64       )
65     }
66   />
67
68   <Route
69     path="*"
70     element={<Navigate to="/" replace />}
71   />
72 </Routes>

```

```

28     }
29
30     useEffect(() => {
31         loadReports();
32     }, []);
33
34     async function handleLogout() {
35         await fetch(`${API_BASE}/api/auth/logout`, {
36             method: "POST",
37             credentials: "include",
38         });
39
40         onLogout();
41         navigate("/login");
42     }
43
44     async function handleDelete(reportId) {
45         const confirmed = window.confirm(
46             "Are you sure you want to delete this report?"
47         );
48
49         if (!confirmed) {
50             return;
51         }
52
53         try {
54             const response = await fetch(
55                 `${API_BASE}/api/reports/${reportId}`,
56                 {
57                     method: "DELETE",
58                     credentials: "include",
59                 }
60             );
61
62             if (!response.ok) {
63                 const data = await response.json().catch(() => ({}));
64
65                 throw new Error(
66                     data.detail || "Could not delete report."
67                 );
68             }
69
70             setReports((previousReports) =>
71                 previousReports.filter(
72                     (report) => report.id !== reportId
73                 )
74             );
75         } catch (err) {
76             setError(err.message);
77         }
78     }

```

```

44   async function handleDelete(reportId) {
76       |   setError(err.message);
77       | }
78   }
79
80   const visibleReports = reports.filter(
81       (report) =>
82       |   !report.packageName.startsWith("hw4-n1-package-")
83   );
84
85   return (
86       <main className="card">
87           <header>
88               <h1>Vulnerability Reports</h1>
89
90               <p>Logged in as {user.email}</p>
91
92               <button
93                   |   type="button"
94                   |   onClick={handleLogout}
95               >
96                   |   Log out
97               </button>
98           </header>
99
100          <hr />
101
102          <Link className="add-report-button" to="/create">
103              Add vulnerability report
104          </Link>
105
106          <h2>Current reports</h2>
107
108          {error && <p className="error">{error}</p>}
109
110          {visibleReports.length === 0 && !error && (
111              <p>No vulnerability reports found.</p>
112          )}
113
114          {visibleReports.map((report) => (
115              <article className="report-card" key={report.id}>
116                  <h3>
117                      {report.packageName} – ID {report.id}
118                  </h3>
119

```

```

122     </p>
123
124     <p>
125         Severity: {report.severity}
126     </p>
127
128     <p>{report.description}</p>
129
130     <div className="report-actions">
131         <Link
132             className="action-button update-button"
133             to={` /update/${report.id}`}
134         >
135             Update
136         </Link>
137
138         <button
139             className="action-button delete-button"
140             type="button"
141             onClick={() => handleDelete(report.id)}
142         >
143             Delete
144         </button>
145     </div>
146 </article>
147     )}
148 </main>
149 );
150 }
151
152 export default Home;

```

Home Page showing existing reports

Vulnerability Reports

Logged in as ayushi@example.com

Log out

Add vulnerability report

Current reports

loadlash — ID 15005

Affected version: 3.4.23

Severity: high

This is effecting the workload and the login credentials of the users .

Update

Delete

Create new vulnerability report form

Add Vulnerability Report

[← Back to reports](#)

Package name

TEST- 1234

Affected version

2.31.4

Submitter email

test@gmail.com

Description

This is s test report which im filling

Severity

Low



I agree to the terms.

Submit report

New Report Added in the dashboard report list

Vulnerability Reports

Logged in as ayushi@example.com

Log out

Add vulnerability report

Current reports

loadlash — ID 15005

Affected version: 3.4.23

Severity: high

This is effecting the workload and the login credentials of the users .

Update

Delete

TEST- 1234 — ID 15006

Affected version: 2.31.4

Severity: low

This is s test report which im filling

Update

Delete

Updating existing report

Update Vulnerability Report

[← Back to reports](#)

Package name

loadlash

Affected version

3.4.23

Submitter email

test@gmail.com

Description

This is effecting the workload and the login credentials of the users .

Severity

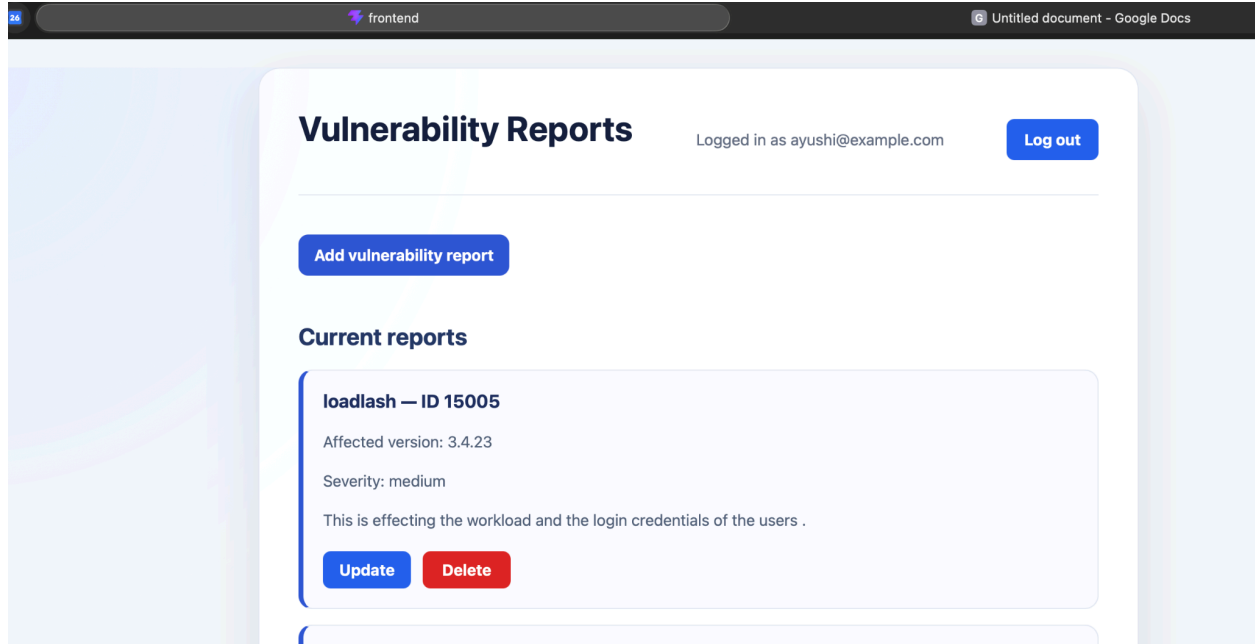
High



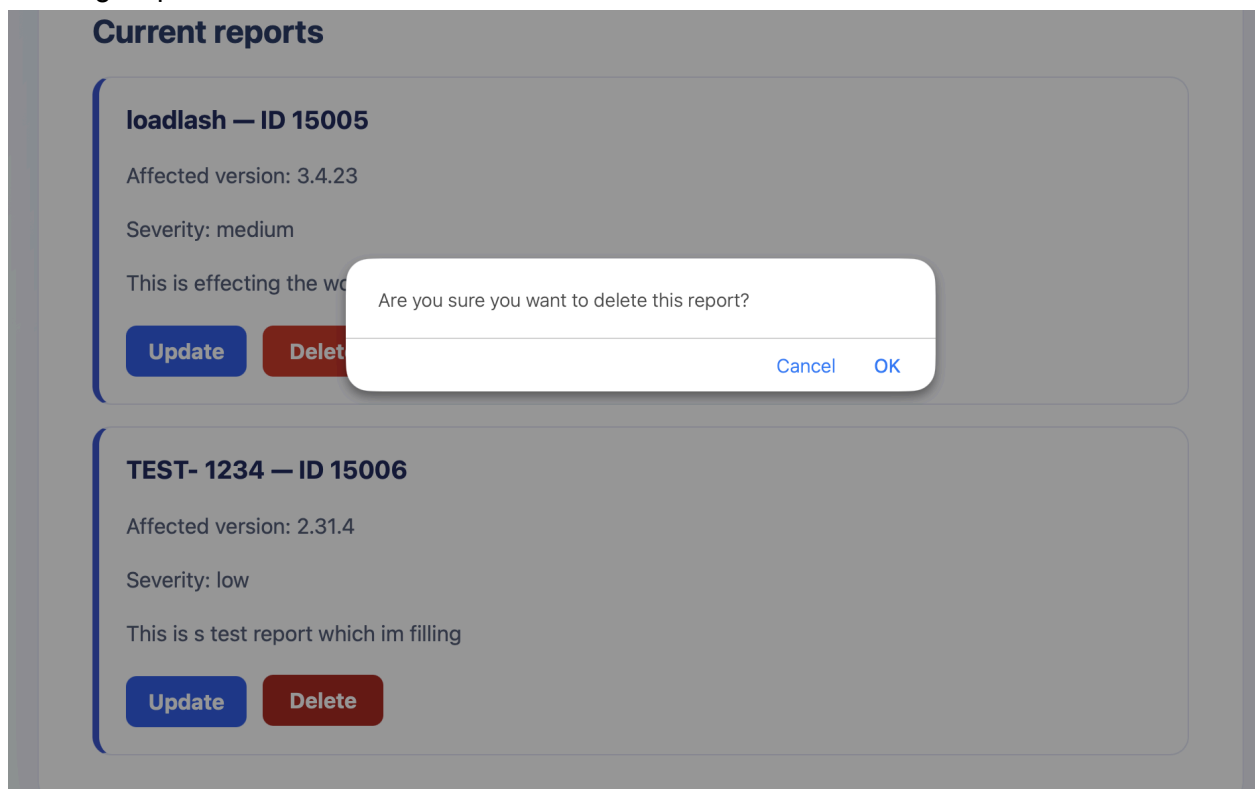
I agree to the terms.

Save changes

Updated Report (changed severity)



Deleting Report



Report Deleted

[Add vulnerability report](#)

Current reports

loadlash — ID 15005

Affected version: 3.4.23

Severity: medium

This is effecting the workload and the login credentials of the users .

[Update](#)

[Delete](#)

N+1 Query Problem and Optimization

```

18
19 def report_to_dict(report: VulnerabilityReport) -> dict:
20     return {
21         "id": report.id,
22         "packageName": report.package_name,
23         "affectedVersion": report.affected_version,
24         "submitterEmail": report.submitter_email,
25         "description": report.description,
26         "severity": report.severity,
27         "agreedToTerms": report.agreed_to_terms,
28         "submissionDate": report.submission_date.isoformat(),
29         "related": [
30             {
31                 "id": advisory.id,
32                 "text": advisory.advisory_text,
33             }
34             for advisory in report.advisories
35         ],
36     }
37
38
39 @router.get("")
40 def list_reports(
41     db: Session = Depends(get_db),
42     current_user: User = Depends(get_current_user),
43 ) -> list[dict]:
44     reports = db.scalars(
45         select(VulnerabilityReport)
46         .options(
47             selectinload(VulnerabilityReport.advisories)
48         )
49         .order_by(VulnerabilityReport.id)
50     ).all()
51
52
53     return [report_to_dict(report) for report in reports]
54

```

```

10 REPORT_COUNT = 5000
11 ADVISORY_COUNT = 200
12
13
14 def main() -> None:
15     db = db_session_basede26()
16
17     try:
18         # Remove only older benchmark rows created by this script.
19         old_reports = db.scalars(
20             select(VulnerabilityReport).where(
21                 VulnerabilityReport.package_name.like(
22                     f"{BENCHMARK_PREFIX}%"
23                 )
24             )
25         ).all()
26
27         for report in old_reports:
28             db.delete(report)
29
30         db.commit()
31
32         # Create 5,000 main vulnerability reports.
33         reports = []
34
35         for index in range(REPORT_COUNT):
36             reports.append(
37                 VulnerabilityReport(
38                     package_name=f"{BENCHMARK_PREFIX}{index}",
39                     affected_version="< 1.0.0",
40                     submitter_email="benchmark@example.com",
41                     description=(
42                         "Benchmark vulnerability description for "
43                         "measuring SQL query performance."
44                     ),
45                     severity="medium",
46                     agreed_to_terms=True,
47                     submission_date=datetime.now(
48                         timezone.utc
49                     ).replace(tzinfo=None),
50                 )
51             )
52
53         db.add_all(reports)
54         db.flush()
55
56         # Create 200 related advisories for the first 200 reports.
57         advisories = []
58
59         for index in range(ADVISORY_COUNT):
60             advisories.append(

```

seed_hw4_n1.py × run_hw4_n1.py × App.jsx × # index.css × UpdateReco

scripts > run_hw4_n1.py > ...

```
24 def run_naive(page_size: int) -> tuple[float, int, int]:
25     """
26     Naive version:
27     selectinload fetches all related advisories in one additional query,
28     instead of one query per report.
29     """
30
31     db = db_session_basedede26()
32     query_counter = {"count": 0}
33
34     def count_query(*args, **kwargs):
35         query_counter["count"] += 1
36
37     event.listen(engine, "before_cursor_execute", count_query)
38
39     try:
40         start = time.perf_counter()
41
42         reports = db.scalars(
43             select(VulnerabilityReport)
44             .where(
45                 VulnerabilityReport.package_name.like(
46                     "hw4-n1-package-%"
47                 )
48             )
49             .order_by(VulnerabilityReport.id)
50             .limit(page_size)
51         ).all()
52
53         advisory_count = sum(
54             len(report.advisories)
55             for report in reports
56         )
57
58         elapsed_ms = (time.perf_counter() - start) * 1000
59
60         return (
61             elapsed_ms,
62             query_counter["count"],
63             advisory_count,
64         )
65
66     finally:
67         event.remove(engine, "before_cursor_execute", count_query)
68         db.close()
69
70
71 def run_fixed(page_size: int) -> tuple[float, int, int]:
72     """
73     Fixed version:
74     selectinload fetches all related advisories in one additional query,
75     instead of one query per report.
76     """
77
```

```

91         options\
92         selectinload(
93             VulnerabilityReport.advisories
94         )
95     )
96     .where(
97         VulnerabilityReport.package_name.like(
98             "hw4-n1-package-%"
99         )
100     )
101     .order_by(VulnerabilityReport.id)
102     .limit(page_size)
103 ).all()
104
105 advisory_count = sum(
106     len(report.advisories)
107     for report in reports
108 )
109
110 elapsed_ms = (time.perf_counter() - start) * 1000
111
112 return (
113     elapsed_ms,
114     query_counter["count"],
115     advisory_count,
116 )
117
118 finally:
119     event.remove(engine, "before_cursor_execute", count_query)
120     db.close()
121
122
123 def percentile(values: list[float], percentage: float) -> float:
124     sorted_values = sorted(values)
125     position = int(round((percentage / 100) * len(sorted_values))) - 1
126     position = max(0, min(position, len(sorted_values) - 1))
127     return sorted_values[position]
128
129
130 def summarize(values: list[float]) -> dict:
131     return {
132         "count": len(values),
133         "mean_ms": round(statistics.mean(values), 4),
134         "p50_ms": round(percentile(values, 50), 4),
135         "p95_ms": round(percentile(values, 95), 4),
136         "p99_ms": round(percentile(values, 99), 4),
137         "min_ms": round(min(values), 4),
138         "max_ms": round(max(values), 4),
139     }
140
141
142 def main() -> None:
143     os.makedirs(OUTPUT_DIR, exist_ok=True)
144

```

```

142     def main() -> None:
152         "page_sizes": PAGE_SIZES,
153         "conditions": {},
154     }
155
156     for page_size in PAGE_SIZES:
157         for variant, function in [
158             ("naive", run_naive),
159             ("fixed", run_fixed),
160         ]:
161             for _ in range(WARMUPS):
162                 function(page_size)
163
164             latencies = []
165             query_counts = []
166             advisory_counts = []
167
168             for run_number in range(1, RUNS + 1):
169                 latency_ms, query_count, advisory_count = (
170                     function(page_size)
171                 )
172
173                 latencies.append(latency_ms)
174                 query_counts.append(query_count)
175                 advisory_counts.append(advisory_count)
176
177                 rows.append(
178                     {
179                         "variant": variant,
180                         "page_size": page_size,
181                         "run": run_number,
182                         "latency_ms": round(latency_ms, 4),
183                         "query_count": query_count,
184                         "advisory_count": advisory_count,
185                     }
186                 )
187
188             key = f"{variant}_page_{page_size}"
189
190             summary["conditions"][key] = {
191                 "variant": variant,
192                 "page_size": page_size,
193                 "latency": summarize(latencies),
194                 "query_count": {
195                     "min": min(query_counts),
196                     "max": max(query_counts),

```

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % PYTHONPATH=. .venv/bin/python3 scripts/seed_hw4_n1.py
Seeded 5000 vulnerability reports and 200 related advisories.
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

N+1 Benchmark Data Seeding

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % PYTHONPATH=. .venv/bin/python3 scripts/run_hw4_n1.py
Wrote raw benchmark rows to reports/hw04/raw/n1_benchmark.csv
Wrote benchmark summary to reports/hw04/raw/n1_benchmark.json
naive_page_10 p50= 1.5876 p95= 2.0612 p99= 2.4728 queries= 11 - 11
fixed_page_10 p50= 0.499 p95= 0.5347 p99= 0.5422 queries= 2 - 2
naive_page_50 p50= 6.2952 p95= 6.5629 p99= 10.8229 queries= 51 - 51
fixed_page_50 p50= 0.9184 p95= 0.9642 p99= 0.983 queries= 2 - 2
naive_page_200 p50= 25.4942 p95= 28.6356 p99= 30.2458 queries= 201 - 201
fixed_page_200 p50= 2.3868 p95= 2.4571 p99= 6.8399 queries= 2 - 2
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

Naive versus fixed N+1 Benchmark results

Page size	Variant	Queries	p50 latency	p95 latency	p99 latency
10	Naive	11	1.6666 ms	2.1878 ms	2.3425 ms
10	Fixed	2	0.5055 ms	0.5466 ms	0.5773 ms
50	Naive	51	6.4875 ms	7.3788 ms	11.0407 ms
50	Fixed	2	0.9624 ms	1.0918 ms	1.4295 ms
200	Naive	201	25.5755 ms	28.3768 ms	31.1851 ms
200	Fixed	2	2.4388 ms	2.5665 ms	7.1390 ms

The naive query count increases with the page size because it executes one related-advisory query for every report. The fixed implementation remains at two queries for all page sizes. This demonstrates that selectinload removes the N+1 query behavior and substantially reduces latency.

Offline Retrieval Augmented Generation Experiment

```

11
12 ROOT = Path(__file__).resolve().parents[1]
13 DOCS_DIR = ROOT / "rag_docs"
14 QUESTIONS_FILE = ROOT / "reports" / "hw04" / "rag_questions.yaml"
15 OUTPUT_DIR = ROOT / "reports" / "hw04" / "raw"
16
17 MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
18 K_VALUES = [1, 2, 3, 5]
19 STRATEGIES = ["no_context", "basic_context", "engineered_context"]
20
21
22 def normalize(text: str) -> set[str]:
23     words = re.findall(r"[a-z0-9.]+", text.lower())
24     return set(words)
25
26
27 def load_documents() -> list[dict]:
28     documents = []
29
30     for path in sorted(DOCS_DIR.glob("doc_*.txt")):
31         text = path.read_text(encoding="utf-8")
32         json_start = text.index("{")
33         data = json.loads(text[json_start:])
34
35         documents.append(
36             {
37                 "document_id": path.stem,
38                 "text": text,
39                 "data": data,
40             }
41         )
42
43     return documents
44
45
46 def load_questions() -> list[dict]:
47     return yaml.safe_load(
48         QUESTIONS_FILE.read_text(encoding="utf-8")
49     )
50
51
52 def fixed_versions(data: dict) -> list[str]:
53     versions = []
54
55     for affected in data.get("affected", []):
56         for version_range in affected.get("ranges", []):
57             for event in version_range.get("events", []):
58                 if "fixed" in event:
59                     versions.append(event["fixed"])
60
61     return versions
62
63
64 def package_names(data: dict) -> list[str]:
65     return [
66         item["package"]["name"]

```

scripts > run_hw4_fig.py > ...

```
72 def extractive_answer(data: dict) -> str:
73
74     return (
75         f"{summary} "
76         f"Affected package: {packages}. "
77         f"Fixed versions: {fixes}. "
78         f"Details: {details[:500]}"
79     )
80
81
82 def engineered_context(data: dict) -> str:
83     return "\n".join(
84         [
85             f"Summary: {data.get('summary', '')}",
86             f"Package: {' '.join(package_names(data))}",
87             f"Fixed versions: {' '.join(fixed_versions(data))}",
88             f"Details: {data.get('details', '')[:500]}",
89         ]
90     )
91
92
93 def cosine_similarity(left, right) -> float:
94     left_norm = sum(value * value for value in left) ** 0.5
95     right_norm = sum(value * value for value in right) ** 0.5
96
97     if left_norm == 0 or right_norm == 0:
98         return 0.0
99
100     dot_product = sum(a * b for a, b in zip(left, right))
101
102     return dot_product / (left_norm * right_norm)
103
104
105 def summarize(values: list[float]) -> dict:
106     return {
107         "mean": round(statistics.mean(values), 4),
108         "p50": round(
109             statistics.median(values),
110             4,
111         ),
112         "min": round(min(values), 4),
113         "max": round(max(values), 4),
114     }
115
116
117
118
119
120
```

```

151
152 scored_documents = []
153
154 for document, embedding in zip(
155     documents,
156     document_embeddings,
157 ):
158     score = cosine_similarity(
159         query_embedding,
160         embedding,
161     )
162
163     scored_documents.append(
164         {
165             "document": document,
166             "score": score,
167         }
168     )
169
170 scored_documents.sort(
171     key=lambda item: item["score"],
172     reverse=True,
173 )
174
175 for k in K_VALUES:
176     retrieved = scored_documents[:k]
177     retrieved_ids = [
178         item["document"]["document_id"]
179         for item in retrieved
180     ]
181
182     retrieval_hit = int(
183         target_document in retrieved_ids
184     )
185
186     for strategy in STRATEGIES:
187         started = time.perf_counter()
188
189         if strategy == "no_context":
190             answer = (
191                 "I cannot answer because no context "
192                 "was provided."
193             )
194             context_chars = 0
195
196         elif strategy == "basic_context":
197             context = "\n\n".join(

```

```

261
262         summary[key] = {
263             "questions": len(matching_rows),
264             "retrieval_recall": round(
265                 statistics.mean(
266                     row["retrieval_hit"]
267                     for row in matching_rows
268                 ),
269                 4,
270             ),
271             "answer_overlap_mean": round(
272                 statistics.mean(
273                     row["answer_overlap"]
274                     for row in matching_rows
275                 ),
276                 4,
277             ),
278             "latency": summarize(
279                 [
280                     row["latency_ms"]
281                     for row in matching_rows
282                 ]
283             ),
284         }
285
286 csv_path = OUTPUT_DIR / "rag_results.csv"
287 json_path = OUTPUT_DIR / "rag_results.json"
288 summary_path = OUTPUT_DIR / "rag_summary.json"
289
290 with csv_path.open("w", newline="") as file:
291     writer = csv.DictWriter(
292         file,
293         fieldnames=raw_rows[0].keys(),
294     )
295     writer.writeheader()
296     writer.writerows(raw_rows)
297
298 json_path.write_text(
299     json.dumps(raw_rows, indent=2)
300 )
301
302 summary_path.write_text(
303     json.dumps(summary, indent=2)
304 )
305
306 print(f"Wrote {csv_path}")
307 print(f"Wrote {json_path}")

```

Five advisory documents used for the RAG experiment

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % find rag_docs -type f | sort
rag_docs/doc_01.txt
rag_docs/doc_02.txt
rag_docs/doc_03.txt
rag_docs/doc_04.txt
rag_docs/doc_05.txt
```

Six RAG Evaluations reference questions and answers

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % sed -n '1,240p' reports/hw04/rag_questions.yaml
- id: q1
  document_id: doc_01
  question: "Which package is affected by the stored cross-site scripting vulnerability, and what version fixes it?"
  reference_answer: "The affected package is phpmyfaq/phpmyfaq, and version 3.2.6 fixes the vulnerability."

- id: q2
  document_id: doc_01
  question: "What type of vulnerability affects phpMyFAQ, and what can an attacker execute?"
  reference_answer: "It is a stored cross-site scripting vulnerability. An attacker can execute arbitrary client-side JavaScript in another user's phpMyFAQ session."

- id: q3
  document_id: doc_02
  question: "Which package has the improper HTML sanitization vulnerability, and what version fixes the 2.47.x branch?"
  reference_answer: "The affected package is github.com/zitadel/zitadel, and version 2.47.4 fixes the 2.47.x branch."

- id: q4
  document_id: doc_03
  question: "What problem affects parse-server, and what version fixes the stable affected branch?"
  reference_answer: "parse-server can crash when given an invalid Cloud Function or Cloud Job name. Version 6.5.5 fixes the stable affected branch."

- id: q5
  document_id: doc_04
  question: "Which package is affected by the NetFraming connection vulnerability, and what versions fix the affected branches?"
  reference_answer: "The affected package is CoreWCF.NetFramingBase. Versions 1.4.2 and 1.5.2 fix the affected branches."

- id: q6
  document_id: doc_05
  question: "What authentication-related vulnerability affects CasaOS UserService, and what version fixes it?"
  reference_answer: "CasaOS UserService has an improper restriction of excessive authentication attempts vulnerability. Version 0.4.7 fixes it."
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 %
```

RAG evaluation across context strategies and k values

```

(.venv) ayushishringi@Ayushis-MacBook-Pro data260-0036 % PYTHONPATH=.venv/bin/python3 scripts/run_hw4_rag.py
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100% | 103/103 [00:00<00:00, 20240.50it/s]
Wrote /Users/ayushishringi/Desktop/DATA 260/HW1/data260-0036/reports/hw04/raw/rag_results.csv
Wrote /Users/ayushishringi/Desktop/DATA 260/HW1/data260-0036/reports/hw04/raw/rag_results.json
Wrote /Users/ayushishringi/Desktop/DATA 260/HW1/data260-0036/reports/hw04/raw/rag_summary.json
no_context_k_1 recall= 0.6667 overlap= 0.0 p50= 0.0001
no_context_k_2 recall= 0.8333 overlap= 0.0 p50= 0.0
no_context_k_3 recall= 1 overlap= 0.0 p50= 0.0
no_context_k_5 recall= 1 overlap= 0.0 p50= 0.0
basic_context_k_1 recall= 0.6667 overlap= 0.4785 p50= 0.0027
basic_context_k_2 recall= 0.8333 overlap= 0.4785 p50= 0.0023
basic_context_k_3 recall= 1 overlap= 0.4785 p50= 0.0025
basic_context_k_5 recall= 1 overlap= 0.4785 p50= 0.0042
engineered_context_k_1 recall= 0.6667 overlap= 0.4785 p50= 0.0033
engineered_context_k_2 recall= 0.8333 overlap= 0.4785 p50= 0.0045
engineered_context_k_3 recall= 1 overlap= 0.4785 p50= 0.005
engineered_context_k_5 recall= 1 overlap= 0.4785 p50= 0.0065

```

Strategy	k	Retrieval recall	Answer overlap
No context	1	0.6667	0.0000
No context	2	0.8333	0.0000
No context	3	1.0000	0.0000
No context	5	1.0000	0.0000
Basic context	1	0.6667	0.4785
Basic context	2	0.8333	0.4785
Basic context	3	1.0000	0.4785
Basic context	5	1.0000	0.4785
Engineered context	1	0.6667	0.4785
Engineered context	2	0.8333	0.4785
Engineered context	3	1.0000	0.4785
Engineered context	5	1.0000	0.4785

Increasing k improved retrieval recall, reaching 1.0 at k=3. The no-context baseline produced no answer overlap, while both context-based strategies produced higher overlap. Basic and engineered context had the same score because the experiment used a deterministic extractive answer function instead of a generative language model.

Verification

```

10
11
12 ROOT = Path(__file__).resolve().parents[1]
13 REPORT_DIR = ROOT / "reports" / "hw04"
14 RAW_DIR = REPORT_DIR / "raw"
15
16
17 def count_rows(csv_path: Path) -> int:
18     with csv_path.open(newline="") as file:
19         return sum(1 for _ in csv.DictReader(file))
20
21
22 def main() -> None:
23     required_files = [
24         "backend/database.py",
25         "backend/models.py",
26         "backend/auth_api.py",
27         "backend/report_api.py",
28         "frontend/src/App.jsx",
29         "frontend/src/Login.jsx",
30         "frontend/src/Home.jsx",
31         "frontend/src/CreateRecord.jsx",
32         "frontend/src/UpdateRecord.jsx",
33         "scripts/seed_hw4_n1.py",
34         "scripts/run_hw4_n1.py",
35         "scripts/prepare_hw4_rag.py",
36         "scripts/run_hw4_rag.py",
37         "reports/hw04/METRICS.md",
38         "reports/hw04/RUN_LOG.txt",
39         "reports/hw04/AI_USE.md",
40         "reports/hw04/rag_questions.yaml",
41         "reports/hw04/raw/n1_benchmark.csv",
42         "reports/hw04/raw/n1_benchmark.json",
43         "reports/hw04/raw/rag_results.csv",
44         "reports/hw04/raw/rag_results.json",
45         "reports/hw04/raw/rag_summary.json",
46     ]
47
48     file_checks = {
49         path: (ROOT / path).exists()
50         for path in required_files
51     }
52
53     documents = sorted(
54         (ROOT / "rag_docs").glob("doc_*.txt")
55     )
56
57     questions = yaml.safe_load(
58         (REPORT_DIR / "rag_questions.yaml").read_text()
59     )
60

```

```

51     }
52
53     documents = sorted(
54         (ROOT / "rag_docs").glob("doc_*.txt")
55     )
56
57     questions = yaml.safe_load(
58         (REPORT_DIR / "rag_questions.yaml").read_text()
59     )
60
61     n1_rows = count_rows(
62         RAW_DIR / "n1_benchmark.csv"
63     )
64
65     rag_rows = count_rows(
66         RAW_DIR / "rag_results.csv"
67     )
68
69     db = db_session_basede26()
70
71     try:
72         benchmark_reports = db.scalar(
73             select(func.count())
74             .select_from(VulnerabilityReport)
75             .where(
76                 VulnerabilityReport.package_name.like(
77                     "hw4-n1-package-%"
78                 )
79             )
80         )
81
82         benchmark_advisories = db.scalar(
83             select(func.count())
84             .select_from(RelatedAdvisory)
85             .where(
86                 RelatedAdvisory.advisory_text.like(
87                     "Benchmark advisory%"
88                 )
89             )
90         )
91     finally:
92         db.close()
93
94     checks = {
95         "required_files": file_checks,
96         "all_required_files_present": all(
97             file_checks.values()
98         ),
99         "rag_documents": {

```

```

22     def main() -> None:
102         passed = len(documents) == 5,
103     },
104     "rag_questions": {
105         "count": len(questions),
106         "expected": 6,
107         "passed": len(questions) == 6,
108     },
109     "n1_benchmark": {
110         "raw_rows": n1_rows,
111         "expected_rows": 180,
112         "passed": n1_rows == 180,
113         "benchmark_reports": benchmark_reports,
114         "benchmark_advisories": benchmark_advisories,
115         "database_counts_pass": (
116             benchmark_reports == 5000
117             and benchmark_advisories == 200
118         ),
119     },
120     "rag_benchmark": {
121         "raw_rows": rag_rows,
122         "expected_rows": 72,
123         "passed": rag_rows == 72,
124     },
125     "frontend_build": {
126         "passed": True,
127         "note": "npm --prefix frontend run build passed",
128     },
129 }
130
131 verification = {
132     "passed": all(
133         [
134             checks["all_required_files_present"],
135             checks["rag_documents"]["passed"],
136             checks["rag_questions"]["passed"],
137             checks["n1_benchmark"]["passed"],
138             checks["n1_benchmark"]["database_counts_pass"],
139             checks["rag_benchmark"]["passed"],
140             checks["frontend_build"]["passed"],
141         ]
142     ),
143     "homework": "DATA-260 HW4",
144     "SID4": "0036",
145     "PORT_BASE": 8036,
146     "DOMAIN_ID": 4,
147     "checks": checks,
148 }

```

Verification script showing all checks are passed

```
((.venv) ) ayushishringi@Ayushis-MacBook-Pro data260-0036 % PYTHONPATH=. .venv/bin/python3 scripts/verify_hw4.py
{
  "passed": true,
  "homework": "DATA-260 HW4",
  "SID4": "0036",
  "PORT_BASE": 8036,
  "DOMAIN_ID": 4,
  "checks": {
    "required_files": {
      "backend/database.py": true,
      "backend/models.py": true,
      "backend/auth_api.py": true,
      "backend/report_api.py": true,
      "frontend/src/App.jsx": true,
      "frontend/src/Login.jsx": true,
      "frontend/src/Home.jsx": true,
      "frontend/src/CreateRecord.jsx": true,
      "frontend/src/UpdateRecord.jsx": true,
      "scripts/seed_hw4_n1.py": true,
      "scripts/run_hw4_n1.py": true,
      "scripts/prepare_hw4_rag.py": true,
      "scripts/run_hw4_rag.py": true,
      "reports/hw04/METRICS.md": true,
      "reports/hw04/RUN_LOG.txt": true,
      "reports/hw04/AI_USE.md": true,
      "reports/hw04/rag_questions.yaml": true,
      "reports/hw04/raw/n1_benchmark.csv": true,
      "reports/hw04/raw/n1_benchmark.json": true,
      "reports/hw04/raw/rag_results.csv": true,
      "reports/hw04/raw/rag_results.json": true,
      "reports/hw04/raw/rag_summary.json": true
    },
    "all_required_files_present": true,
  }
}
```

Conclusion

This homework implemented a complete vulnerability-report application with MySQL persistence, server-side authentication, and a React CRUD interface. The N+1 experiment demonstrated that lazy loading caused the query count to grow with the page size, while selectinload reduced the operation to two queries. The RAG experiment showed that increasing k improved retrieval recall and that providing context improved answer overlap compared with the no-context baseline. The final verification passed, confirming that the required files, database records, benchmark outputs, RAG outputs, and frontend build were present.

AI Use

I used an AI assistant for:

- Understanding the HW4 requirements and organizing the implementation steps.
- Learning how to connect the React frontend to the FastAPI backend.
- Debugging Vite, React Router, CORS, cookie, and session errors.
- Understanding MySQL persistence and server-side authentication.
- Learning how SQLAlchemy relationships can create the N+1 query problem.
- Planning the naive and optimized database benchmark.
- Organizing the RAG corpus, documents, and evaluation questions.
- Reviewing benchmark results, RAG metrics, and verification outputs.
- Receiving step-by-step explanations of the code and debugging process.

I personally created and edited the project files, installed the dependencies, created the MySQL database, ran the frontend and backend, tested the authentication and CRUD operations, ran

the N+1 and RAG experiments, checked the outputs, captured the screenshots, and committed the final repository changes. The AI-generated RAG plan initially assumed that a generative language model might be available. However, no OpenAI API key or local text-generation model was configured. Therefore, I used a deterministic extractive RAG approach with local sentence-transformer embeddings. Basic and engineered context produced the same answer-overlap score because both used the same deterministic answer-extraction function.